Name : Jay Moradiya
Roll No : 25AIML034

Practical – 11

Object Oriented Programming (CEUE203)

AIM:

To implement file handling, object serialization, transaction logging and report generation using Java I/O and NIO.

Part A – Practice Programs (Hour 1)

1. **Save/Load using Serialization**

   Create a class that implements Serializable, save an array of its objects to a file using ObjectOutputStream, then read it back using ObjectInputStream. Mark one field as transient and show that it is not saved.

Program:

```
import java.io.*;

class Student implements Serializable {
    private static final long serialVersionUID = 1L;
    int id;
    String name;
    transient String temp;   // not saved

    Student(int id, String name, String temp) {
        this.id = id;
        this.name = name;
        this.temp = temp;
    }

    public String toString() {
        return "ID: " + id + ", Name: " + name + ", Temp: " + temp;
    }
}

public class SaveLoadDemo {
    public static void main(String[] args) throws Exception {
        Student[] arr = {
            new Student(1, "Jay", "data1"),
            new Student(2, "Om",  "data2")
        };

    ObjectOutputStream out = new ObjectOutputStream(new FileOutputStream("students.dat"));
    out.writeObject(arr);
    out.close();
    ObjectInputStream in = new ObjectInputStream(new FileInputStream("students.dat"));
    Student[] read = (Student[]) in.readObject();
    in.close();
    for (Student s : read)
}       System.out.println(s);
```

Output:

```
ID: 1, Name: Jay, Temp: null
ID: 2, Name: Om, Temp: null
```

2. Log Analyzer

Create few text files and read each line using Path/Files. Count total lines and how many contain a keyword. Also print each file's size.

Program:

```
import java.nio.file.*;
import java.nio.file.attribute.*;
import java.io.*;

public class LogAnalyzer {
    public static void main(String[] args) throws Exception {
     Path folder = Paths.get("logsA");
     Files.createDirectories(folder);

     Files.writeString(folder.resolve("log1.txt"), "Deposit 100\nWithdraw 50\nDeposit 200");
     Files.writeString(folder.resolve("log2.txt"), "Withdraw 30\nDeposit 400");

     int totalLines = 0, keywordCount = 0;
     String keyword = "Deposit";

     try (DirectoryStream>Path> stream = Files.newDirectoryStream(folder, "*.txt")) {
         for (Path file : stream) {
          long size = Files.size(file);
          System.out.println("File: " + file.getFileName() + " | Size: " + size + " bytes");

          for (String line : Files.readAllLines(file)) {
              totalLines++;
              if (line.contains(keyword) keywordCount++;
          }
     }

     System.out.println("Total Lines: " + totalLines);
     System.out.println("Lines with '"+ keyword +"': " + keywordCount);
}  }
}
```

Output:

```
File: log1.txt | Size: 45 bytes
File: log2.txt | Size: 32 bytes
Total Lines: 5
Lines with 'Deposit': 3
```

3. Folder Walk (Advanced – Home)

Walk a folder tree using NIO and produce a report of files with name, size and modified time.

Program:

```
import java.nio.file.*;
import java.nio.file.attribute.*;
import java.io.*;
import java.time.*;

public class FolderWalk {
    publit static void main(String[] args) throws Exception {
      Path start = Paths.get(".");
      Path report = Paths.get("folder_report.txt");

      try (BufferedWriter writer = Files.newBufferedWriter(report)) {
          Files.walk(start)
           -filter(Files::isRegularFile)
           -forEach(p -> {
              try {
               BasicFileAttributes a = Files.readAttributes(p, BasicFileAttributes.class);
               writer.write(p.toString() +"|"+ a.size() +" bytes | "+ a.lastModifiedTime());
               writer.newLine();
              I catch (Exceptien e) {}
          });
      }
      System.out.println("Report generated in folder_report.txt");
}
```